L09 — Full Timed Instructor Script

110-minute plan with 90-minute compression path

GenAI Basics · Module 09 · 2026-09-08

Session timing

Block	Minutes	Slides	Purpose
Opening and objectives	0–8	1–3	Set the mental model: Claude as workbench, not chatbot.
Ecosystem and surface selection	8–25	4–5	Give students a selection map they can use immediately.
Artifacts concept and prompt pattern	25–45	6–9	Teach artifact-first prompting and acceptance tests.
Live demo	45–72	10–11	Build and revise a support triage dashboard.
Developer layer	72–93	12–15	Connect artifacts to API building blocks, tools, files, citations, caching and Skills.
Governance and evaluation	93–105	16–17	Shift from demo quality to deployable quality.
Hands-on briefing and Q&A	105–120	18–22	Assign workshop/homework and close.

Opening script

0:00–0:02 — Show slide 1. Say: “Today we use Claude as a workbench. The goal is not to collect another list of model names. The goal is to produce something reusable, test it, and understand what changes when this moves from a browser tab into a product.”

0:02–0:05 — Ask: “When you ask an AI tool to create a dashboard, what do you normally check first: how it looks, or whether it answers the operational question?” Let two students answer. Bridge: “That reflex is exactly why Artifacts are useful and dangerous. They make output visible, but visibility can trick us into thinking it is correct.”

0:05–0:08 — Slide 3. Walk through outcomes. Emphasize that the deliverable is not a reflection paragraph; students will create a build brief and an inspected artifact.

Slide-by-slide script

Slide	Time	Instructor notes
2	8:00–12:00	Contrast a chat answer with an artifact. A chat answer is convenient; an artifact can be reused, tested, shown to a stakeholder or exported. Use the line: “The artifact is where prompting starts to look like product work.”
4	12:00–19:00	Explain the ecosystem map. Do not go deep into Claude Code yet; that returns in the agentic development module. Keep the decision clear: chat explores, artifacts package, code changes repositories, API integrates, managed agents run workflows, Skills encode repeatable methods.
5	19:00–25:00	Use two examples. Example one: student needs a study summary from a PDF — chat or artifact. Example two: product team needs a customer-support tool inside its portal — API/agent path. Ask students to identify the surface.
6	25:00–30:00	Define artifact criteria. It should be self-contained and useful later. Warn against forcing everything into an artifact; quick answers do not need a side-panel product.
7	30:00–35:00	Teach the loop. Stress that the first artifact is a draft. The professional move is to test it against acceptance criteria and request focused changes.
8	35:00–40:00	Walk through prompt anatomy. Give students permission to be specific. Specific does not mean a five-page prompt; it means the right constraints and tests.
9	40:00–45:00	Read weak and strong prompts. Ask: “Which one can we evaluate?” The answer is the strong one because it has data, task and visible success criteria.
10	45:00–48:00	Introduce demo data: support tickets with severity, age, owner and status. The operational question: what should the support lead act on this morning?
11	48:00–72:00	Run demo. First paste weak prompt and show what is missing. Then paste strong prompt from the runbook. Inspect: stale P1 visibility, filters, summary KPIs, privacy. Ask for one targeted revision. If Claude is unavailable, open the local fallback

		HTML and perform the same inspection.
12	72:00–78:00	Switch mental model from UI to API. Messages are not magic; they are protocol. A tool call is not action until your application or server-side infrastructure executes it.
13	78:00–84:00	Explain evidence workflow. Files reduce repeated uploads; PDFs/documents can be source material; citations create pointers. Caveat: citations point to provided documents, not to truth itself.
14	84:00–89:00	Explain context and caching through the “stable suitcase” metaphor: pack stable instructions once; do not repack the suitcase every turn with timestamps and changing tool definitions.
15	89:00–93:00	Explain Skills as reusable work instructions with resources. Compare a Skill to a carefully maintained internal playbook plus scripts, not to a magic plugin.
16	93:00–99:00	Governance: publishing is publication, organization sharing still requires permissions, and third-party Skills require audit. Say: “The easiest leak is a beautiful artifact that everyone wants to share before anyone asks what data is inside.”
17	99:00–105:00	Artifact evaluation. Have students vote on the six criteria. Emphasize task success and safe reuse over aesthetics.
18	105:00–110:00	Explain workshop steps. Pair students if needed. Output: brief, prompt, acceptance matrix, revised artifact or screenshot/HTML.
19	110:00–114:00	Homework. Show that the brief can feed later modules on open models, agents and Streamlit.
20	114:00–117:00	Common failure modes. Use examples from the demo. Encourage targeted revision instead of “make it better.”
21	117:00–119:00	Summarize key takeaways.
22	119:00–120:00	Q&A and next module bridge.

90-minute compression path

- Keep slides 1–9 to 25 minutes by removing one surface-selection example.
- Run only the strong demo prompt; skip the weak prompt except as a verbal warning.

- Compress developer layer to slides 12–15 in 12 minutes; focus on tools/files/context/Skills and defer Managed Agents to later modules.
- Keep governance and practice intact. Do not cut the acceptance-test exercise.

Likely Q&A

Question	Answer
Can I just ask Claude to make the artifact better?	Yes, but that is weaker than giving a targeted change request tied to an acceptance test.
Are artifacts safe to share?	They can be shared, but sharing mode and audience depend on plan and org settings. Treat public links as publication.
Are Skills like plugins?	Partly, but a Skill is primarily a reusable bundle of instructions/resources/scripts. It should be reviewed like a dependency.
Does prompt caching store my data forever?	No. Caches are temporary and isolated, but details differ by platform; verify current retention rules before production use.
Should we build with Claude API or just use Claude.ai?	Use the web UI for human workflows and prototypes; use the API when the workflow must be integrated, logged, governed or scaled.